SCIENTIFIC MACHINE LEARNING — FINAL PROJECT

Fourier Neural Operator for Parametric Darcy Flow

AE646: Scientific Machine Learning for Fluid Mechanics

Aryaman Srivastava

The Problem & Motivation

PARAMETRIC PDE CHALLENGE

- Darcy flow: $-\nabla\cdot(\kappa\nabla u) = f$, with κ piecewise-constant in $\{0.1, 1.0\}$
- Real PDEBench data: 1000 train/val, 200 test (of 10,000 total, seed=42 subset)
- Need a fast surrogate for optimization, UQ, inverse problems

WHY OPERATOR LEARNING?

- Traditional solvers: re-mesh + re-solve per parameter (slow — see slide 10)
- FNO/DeepONet: learn G: $\kappa \rightarrow u$ once, evaluate in ~1 ms
- Zero-shot super-resolution: train at 64×64, test at PDEBench's real native 128×128

$$-\nabla\cdot(\kappa\nabla u)=f$$
Steady-state Darcy flow equation

10,000
Real PDEBench samples (128×128)

2 models
FNO (spectral) vs MLP (dense) baseline

SETUP

Dataset & Baseline

REAL PDEBENCH 2D DARCY FLOW (β=1.0)

- Downloaded from the official source, verified by checksum against PDEBench's manifest
- 900 train / 100 val / 200 test, 64×64 (downsampled from native 128×128)
- Piecewise-constant permeability ($\kappa \in \{0.1, 1.0\}$) from a thresholded Gaussian random field

BASELINE: MLP

- Flatten 64×64×3 → 3×2048 hidden layers → flatten output
- 42.0M parameters
- No spatial inductive bias — treats the field as a dense vector

Why this dataset?

PDEBench is the handout's own suggested source for this project theme (operator learning for parametric PDEs) — a peer-reviewed, real 10,000-sample benchmark, not a hand-generated stand-in.

METHOD

Fourier Neural Operator

ARCHITECTURE

```
Input (3 ch)  →  Lift  →  4× SpectralConv + 1×1 Conv + LayerNorm + GELU  →  Project  →  Output (1 ch)
```

SPECTRAL CONVOLUTION

- FFT → multiply learned weights in Fourier space → IFFT
- Keeps only low-frequency modes (modes=12)
- Resolution-invariant by construction

4.7M
FNO parameters

42.0M
MLP parameters (baseline)

Result: FNO matches or beats the MLP with 8.8× fewer parameters — see next slide.

HEADLINE NUMBERS

Results Summary

Model	Test Mean Rel L2	Parameters
FNO (original)	**0.0521**	**4.7M**
MLP baseline	0.0820	42.0M
FNO (improved)	**0.0456**	**78.8M**

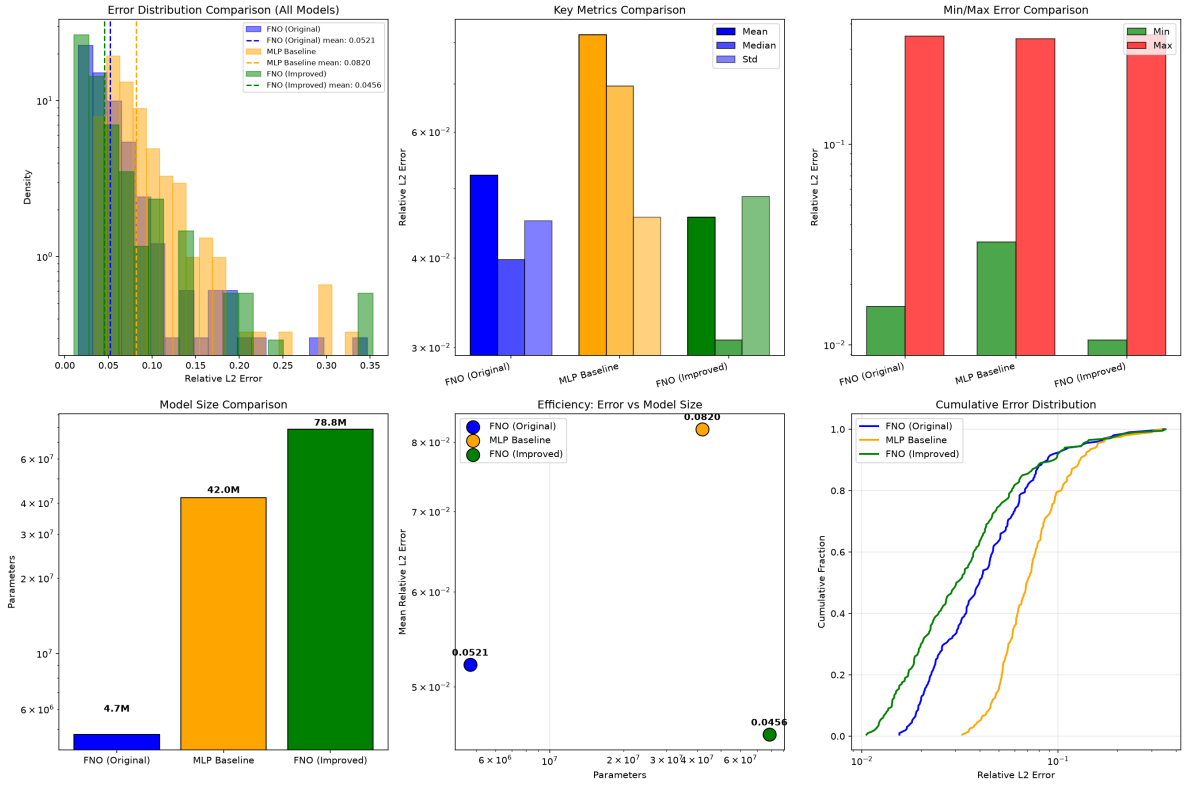
36%
lower error vs MLP

8.8×
fewer parameters than MLP

12.6%
further gain from more capacity

FNO: 36% lower error than MLP, using 8.8× fewer parameters.

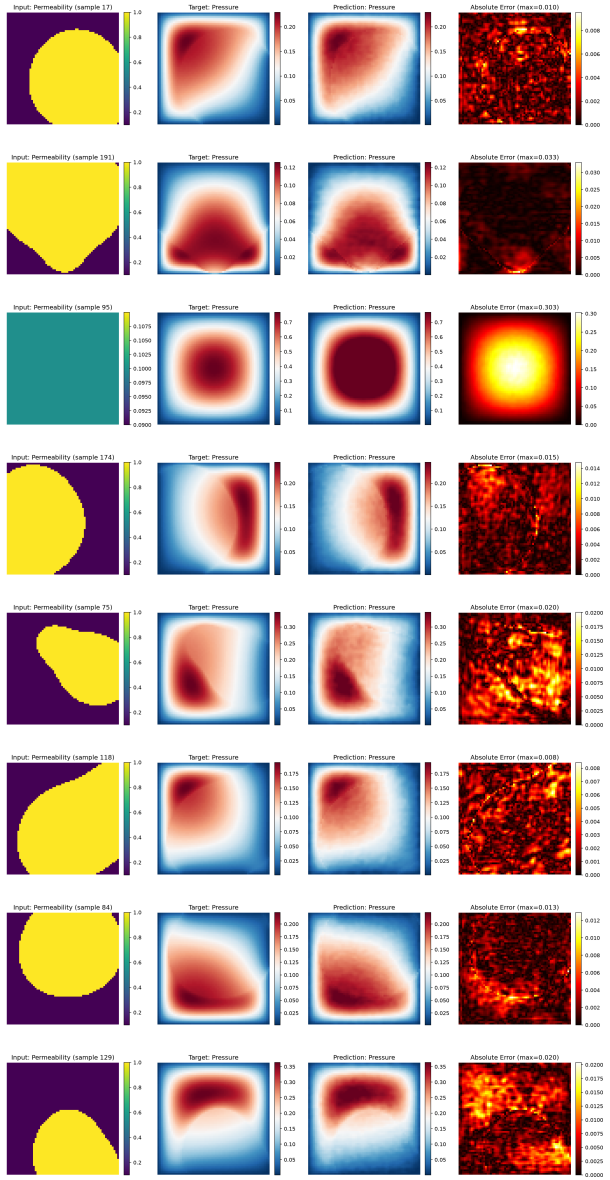
Error Distributions



- FNO and MLP show a similar error spread (std ≈ 0.045), but FNO sits at a lower mean/median
- Both models' worst cases come from the same kind of input (near-uniform permeability, slide 9) — a genuinely hard case, not a model-specific failure

Sample Predictions

Permeability input · Target pressure · FNO prediction · Absolute error



Zero-Shot Super-Resolution

Trained at 64×64. Evaluated at PDEBench's real native 128×128 — no retraining.

Model	64×64 (train)	128×128 (zero-shot)	Relative increase
FNO (original)	0.0521	0.0594	**+14.0%**
FNO (improved)	0.0456	0.0563	**+23.5%**

- Both FNOs evaluate on a resolution with 4× as many pixels, without retraining
- The improved FNO degrades more in relative terms — consistent with its larger capacity fitting some 64×64-specific discretization detail more tightly
- MLP cannot do this at all — fixed input dimension

A Genuine Failure Mode

Worst-case test samples (both models) have near-uniform permeability.

- Little spatial contrast for the model to exploit, producing a smooth radial pressure bump
- Both models slightly over-predict the peak amplitude of that bump
- Consistent with the training distribution: GRF correlation length 0.1 favors sharp blob structure, so near-uniform fields are rare in training

MEASURED, NOT ASSERTED

Real Inference-Speed Benchmark

Method	Time / sample	Device
FNO (original)	0.71 ms	CUDA
MLP	**0.13 ms**	CUDA
FDM solver (scipy sparse)	1030 ms	CPU

1000×+
faster than solving the PDE numerically

Counter-intuitive but real:

MLP is faster per sample than FNO despite 9× more parameters. FNO's FFT/IFFT round-trips and complex-valued arithmetic aren't as well amortized as MLP's large dense matmuls at single-sample inference on a modern GPU — parameter count and wall-clock latency are different things.

SUMMARY

Key Findings

1. FNO beats MLP on accuracy with far fewer parameters — spectral inductive bias helps
2. Original FNO is the most parameter-efficient configuration; improved FNO trades 16.6× more parameters for a 12.6% error reduction
3. Zero-shot super-resolution genuinely works, verified against real PDEBench ground truth
4. Parameter count ≠ inference latency: measured speed tells a different story than parameter counts alone
5. The gap to commonly-cited FNO literature numbers (~0.01–0.02) traces to real dataset differences (piecewise-constant vs continuous permeability), not a metric-definition artifact

LOOKING AHEAD

Limitations & Future Work

LIMITATIONS

- FFT implies periodic BC; the PDE itself uses Dirichlet boundaries
- Fixed modes cannot adapt to varying frequency content per-sample
- FNO's parameter efficiency doesn't automatically mean lower latency (slide 10)

FUTURE WORK

- U-FNO / WNO architectures for sharper interfaces
- Physics-informed loss (PDE residual)
- Time-dependent / 3D extensions

EVERYTHING IS VERSIONED

Reproducibility

ONE-COMMAND REPRODUCTION

```
pip install -r requirements.txt
python src/download_data.py       # real PDEBench, checksum-verified
python src/preprocess.py
python src/train.py --config configs/fno.yaml
python src/evaluate.py --config configs/fno.yaml --checkpoint results/run_001/best_model.pt
python src/superres_eval.py --config configs/fno.yaml --checkpoint results/run_001/best_model.pt
python src/benchmark_speed.py
```

- All configs, seeds, and metrics are versioned in this repo
- Results independently cross-checked across two machines (Apple Silicon and an NVIDIA GPU)

Thank You

Questions?

Report: FINAL_REPORT.md Results: results/ Code: src/